

TAREA ACADÉMICA FINAL DE CURSO

Clave : 1INF61 **Créditos** : 03
Curso : Graph Machine Learning **Semestre** : 2026-1
Docentes : Dr. César A. Beltrán Castañón,
Dr. Edwin Alvarez Mamani

La tarea académica final del curso permitirá que los estudiantes apliquen sus conocimientos en el diseño, implementación y evaluación de modelos de Graph Machine Learning (GML) para resolver un problema específico. El objetivo principal de la actividad será la experimentación con metodologías del estado del arte en GML, incluyendo Graph Neural Networks (GNNs), técnicas de representación de nodos, aristas o grafos completos, aplicando buenas prácticas de investigación, diseño y evaluación experimental. Los proyectos están diseñados para completarse en tres semanas y deberán emplear herramientas, repositorios y librerías especializadas para GML, tales como PyTorch Geometric o Deep Graph Library, Asimismo, se recomienda utilizar datasets públicos de referencia y desarrollar una demostración interactiva que permita visualizar y explorar los resultados obtenidos.

Actividad/Entregable	Fecha de entrega	Evaluación
Selección del tema.	-	-
Entregable 1: Formulación del proyecto - Contexto y dominio de aplicación - Descripción del problema - Objetivos (general y específicos) - Fuente de datos - Modelamiento del problema como grafo - Nivel de análisis en el grafo y tarea de aprendizaje - Trabajos relacionados	19 de junio	2.5 puntos
Entregable 2: Recolección y preprocesamiento de datos - Recolección de datos - Descripción del dataset - Preprocesamiento de datos - Construcción del grafo - Análisis estructural del grafo	19 de junio	2.5 puntos
Entregable 3: Análisis basado en GML (experimentos) - Selección del modelo - Entrenamiento y optimización de hiperparámetros - Evaluación del modelo	26 de junio	5 puntos

Presentación final del proyecto - Exposición - Funcionamiento de la solución - Informe final	03 de julio	10 puntos
--	-------------	-----------

Elección de la propuesta

Cada grupo deberá elegir una aplicación, la cual será comunicado al delegado (a) del curso quien indicará si el tema todavía está disponible, algunas recomendaciones se listan a continuación.

Nº	Paper	Code
1	Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition	https://github.com/yysijie/st-gcn
2	SuperGlue: Learning Feature Matching with Graph Neural Networks	https://github.com/magicleap/SuperGluePretrainedNetwork
3	FraudGT: A Simple, Effective, and Efficient Graph Transformer for Financial Fraud Detection	https://github.com/junhongmit/FraudGT#fraudgt-a-simple-effective-and-efficient-graph-transformer-for-financial-fraud-detection
4	T-GAE: Transferable Graph Autoencoder for Network Alignment	https://github.com/Jason-Tree/T-GAE
5	STGC-GNNs: A GNN-based traffic prediction framework with a spatial-temporal Granger causality graph	https://github.com/lehaifeng/T-GCN/tree/master/STGC-GNN
6	LSTTN: A Long-Short Term Transformer-based Spatio-temporal Neural Network for Traffic Flow Forecasting	https://github.com/GeoX-Lab/LSTTN
7	LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation	https://github.com/gusye1234/LightGCN-PyTorch
8	GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning	https://github.com/cmavro/GNN-RAG
9	ROLAND: Graph Learning Framework for Dynamic Graphs	https://github.com/snap-stanford/ROLAND
10	Position: Relational Deep Learning - Graph Representation Learning on Relational Databases	https://github.com/snap-stanford/relbench

Etapas

1. Formulación del proyecto

El primer entregable debe presentar una descripción inicial del proyecto y del problema que se abordará mediante análisis de grafos usando GML. Como mínimo, el documento debe incluir los siguientes elementos:

1.1. Contexto y dominio de aplicación

Describir el dominio o área donde se aplica el proyecto (por ejemplo, redes sociales, negocios, biología, transporte, fraude, etc.).

1.2. Descripción del problema

Definir de forma precisa el problema a resolver y explicar por qué es relevante dentro del contexto identificado.

1.3. Objetivos (general y específicos)

Establecer el propósito principal que guiará el desarrollo del proyecto.

1.4. Datos y fuentes de información

Descripción preliminar de los datos que se utilizarán o de las posibles fuentes para construir la red.

1.5. Modelamiento del problema como grafo

Descripción de cómo se modelará el sistema utilizando un grafo:

- Qué representan los nodos
- Qué representan las aristas o relaciones
- Tipo de grafo (dirigido/no dirigido, ponderado/no ponderado, etc.)

1.6. Nivel de análisis en el grafo y tarea de aprendizaje

Indicar en qué nivel del grafo se realizará el análisis. Es decir, especificar si el objetivo del modelo es realizar predicciones o inferencias a nivel de nodos, aristas, subgrafos o grafo completo.

Ejemplos:

- Nivel nodo: clasificación de nodos, predicción de atributos de nodos, etc.
- Nivel arista: predicción o clasificación de enlaces, etc.
- Nivel subgrafo: identificación o clasificación de subestructuras dentro del grafo, etc.
- Nivel grafo: clasificación o comparación de grafos completos, etc.

1.7. Trabajos relacionados

Identificar al menos dos trabajos similares (por ejemplo, artículos científicos o papers) que aborden problemas relacionados o utilicen técnicas de *machine learning* con grafos. Se debe incluir una breve descripción de cada trabajo y explicar su relación con el proyecto propuesto.

Este entregable debe servir como base conceptual del proyecto, asegurando que el problema, el enfoque basado en grafos y su aplicación práctica estén claramente definidos.

Instrucciones:

- Subir el informe/documento en .PDF (uno por grupo)
- Para la revisión: el grupo realiza una explicación breve (10 minutos)

2. Recolección y preprocesamiento de datos

Tomando como base la fundación conceptual establecida en el Entregable 1, este entregable se centra en la construcción y preprocesamiento del conjunto de datos que soportará el análisis definido en el proyecto. El documento como mínimo debe incluir los siguientes elementos:

2.1. Recolección de datos

Describir el proceso seguido para recolectar u obtener los datos utilizados para construir el grafo. Esto incluye las fuentes de datos (APIs, datasets públicos, *web scraping*, bases de datos institucionales, etc.), la metodología de recolección y las limitaciones o restricciones encontradas durante este proceso.

2.2. Descripción del dataset

Proporcionar una descripción detallada del *dataset* en su estado crudo, incluyendo el número de registros, atributos, tipos de datos, etc. Justificar por qué este dataset es apropiado para el problema definido en el Entregable 1.

2.3. Preprocesamiento de datos

Documentar todos los pasos de preprocesamiento aplicados al *dataset* previo a la construcción del grafo, incluyendo el manejo de valores faltantes, eliminación de duplicados, normalización o filtrado de atributos, transformación aplicada a los atributos de nodos o aristas, etc. Esta etapa debe dejar listo el dataset para la posterior construcción del grafo.

2.4. Construcción del grafo

Describir cómo fue transformado en una estructura de grafo, manteniendo coherencia con el modelo de grafo definido en el Entregable 1:

- Cómo se identificaron e instanciaron los nodos
- Cómo se asignaron los características o atributos de la nodos, si aplica
- Cómo se identificaron e instanciaron las aristas (relaciones)
- Cómo se asignaron los pesos o atributos de las aristas, si aplica
- Mostrar la información del grafo
- Visualización del grafo

2.5. Análisis estructural del grafo

Presentar un análisis estructural preliminar del grafo construido, incluyendo el cómputo e interpretación de métricas básicas a nivel de nodos, de aristas y de grafo.

Instrucciones:

- Subir el informe/documento en .PDF (debe de contener el entregable 1)
- Subir el código fuente en .ipynb o .py (recomendable código en GitHub)
- Para la revisión: el grupo realiza una explicación breve (10 minutos)

3. Análisis basado en GML

Este enfoque utiliza modelos de machine learning con grafos, particular, *Graph Neural Networks* (GNN) para realizar predicciones, descubrir patrones y aprender representaciones a partir de datos estructurados.

3.1. Seleccionar el modelo adecuado para la tarea y el tipo de grafo

- Elegir el tipo de modelo según el problema:
 - Modelos de embeddings (DeepWalk, node2vec, etc)
 - GNNs (GCN, GAT, GraphSAGE, etc.)
- Determinar la estrategia de validación (*train/val/test* por nodos, aristas o grafos).

3.2. Entrenamiento y optimización de hiperparámetros

- Entrenar el modelo con los datos de la red.
- Optimización de hiperparámetros (tasa de aprendizaje, número de capas, dimensiones del *embedding*, épocas, etc.).
- Monitorear métricas como *precisión*, *accuracy*, *recall*, AUC, pérdida, etc.

3.3. Evaluación

- Evaluar el desempeño del modelo usando métricas de evaluación adecuadas al tipo de tarea/aplicación.
- Comparaciones con la línea base.

Instrucciones:

- Subir el informe/documento en .PDF (debe de contener el entregable 1 y 2)
- Subir el código fuente en .ipynb o .py (recomendable código en GitHub)
- Para la revisión: el grupo realiza una explicación breve (10 minutos)

Recursos

- Buscador de literatura académica
<https://scholar.google.com.pe/>
- Formato IEEE (MS Word y LaTeX)
https://www.ieee.org/conferences_events/conferences/publishing/templates.html
- Editor colaborativo LaTeX en línea
<https://www.overleaf.com/>
- Working with Jupyter Notebook files on GitHub
<https://help.github.com/articles/working-with-jupyter-notebook-files-on-github/>
- Librería Torch Geometric
<https://pytorch-geometric.readthedocs.io/en/latest/>
- Repositorio SNAP
<https://snap.stanford.edu/data/index.html>
- Repositorio Network
<https://networkrepository.com/>